HTML & CSS Nhập Môn

Tài liệu gốc kèm thực hành bổ sung · 29/10/2026

Mục lục

- HTML Cơ Bản: Cấu trúc tài liệu HTML
- HTML Cơ Bản: Form và bảng biểu
- HTML Cơ Bản: Semantic HTML
- CSS Cơ Bản: Selector & thuộc tính CSS
- CSS Cơ Bản: Box Model & Flexbox
- CSS Cơ Bản: CSS Grid
- Responsive Design: Media Query
- Responsive Design: Mobile First Design
- Responsive Design: Dự án: Landing Page Responsive

CHƯƠNG 1 · BÀI 1

1.1. Cấu trúc tài liệu HTML

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu HTML là gì và vai trò của nó
> - Nắm cấu trúc cơ bản của một tài liệu HTML
> - Sử dụng các thẻ tiêu đề, đoạn văn, liên kết, hình ảnh và danh sách

1. HTML là gì

HTML (HyperText Markup Language) là ngôn ngữ đánh dấu dùng để mô tả cấu trúc của trang web. HTML không phải ngôn ngữ lập trình vì không có biến, vòng lặp hay điều kiện. Trình duyệt đọc mã HTML và hiển thị nội dung theo các thẻ (tag) được khai báo.

2. Cấu trúc tài liệu HTML

Dòng khai báo `<!DOCTYPE html>` cho trình duyệt biết tài liệu dùng chuẩn HTML5. Thẻ gốc của mọi trang web là `html`, bên trong gồm hai phần `head` và `body`. Phần `head` chứa thông tin mô tả trang như `title`, `meta charset` và liên kết tới file CSS. Phần `body` chứa toàn bộ nội dung hiển thị cho người dùng.

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Trang đầu tiên</title>
</head>
<body>
  <h1>Xin chào</h1>
  <p>Đây là đoạn văn.</p>
</body>
</html>
```

3. Thẻ và thuộc tính

Một phần tử HTML gồm thẻ mở, nội dung và thẻ đóng, ví dụ `<p>Xin chào</p>`. Thuộc tính (attribute) là thông tin bổ sung đặt trong thẻ mở, ví dụ `href` hoặc `class`. Thẻ tự đóng như `img`, `br` và

`input` không có nội dung và không cần thẻ đóng.

4. Các thẻ thường dùng

Các thẻ tiêu đề từ `h1` đến `h6`, trong đó `h1` là tiêu đề quan trọng nhất của trang. Liên kết được tạo bằng thẻ `a`, địa chỉ đích nằm trong thuộc tính `href`; thuộc tính `target="_blank"` mở liên kết trong tab mới. Hình ảnh chèn bằng thẻ `img`, đường dẫn ảnh nằm trong `src` và mô tả thay thế nằm trong `alt`. Danh sách không thứ tự dùng `ul`, có thứ tự dùng `ol`, mỗi mục là một thẻ `li`.

```
<a href="https://example.com" target="_blank">Trang chủ</a>
<img src="logo.png" alt="Logo StudyOnline">
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
```

Tóm tắt

- HTML mô tả cấu trúc trang; `head` chứa thông tin, `body` chứa nội dung.
- Thuộc tính đặt trong thẻ mở; `a` dùng `href`, `img` dùng `src` và `alt`.
- `ul`/`ol` với `li` tạo danh sách.

Câu hỏi ôn tập và bài tập vận dụng

1. Tạo trang giới thiệu bản thân gồm tiêu đề, đoạn văn, một ảnh và một danh sách sở thích.
2. Thêm liên kết tới một trang web và cho mở trong tab mới.
3. Giải thích vì sao thuộc tính `alt` của ảnh quan trọng.

Cấu trúc tài liệu HTML — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: tiếng Việt lỗi hoặc trang co sai trên điện thoại. Dùng tài liệu HTML có charset UTF-8, lang phù hợp và viewport; lưu file đúng encoding.

```
<!doctype html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Khóa học của tôi</title>
</head>
<body><main><h1>Khóa học</h1></main></body>
</html>
```

Tự luyện: đổi title và kiểm tra tên tab, thứ bậc heading và ký tự có dấu. **Hướng giải:** title mô tả trang; h1 mô tả nội dung chính; không dùng tiêu đề chỉ để chọn cỡ chữ.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development

CHƯƠNG 1 · BÀI 2

1.2. Form và bảng biểu

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

> ### Mục tiêu bài học
>
> - Tạo bảng biểu bằng các thẻ table, tr, th, td
> - Xây dựng form nhập liệu với các kiểu input phổ biến
> - Gắn nhãn cho ô nhập liệu và hiểu cách form gửi dữ liệu

1. Bảng biểu

Bảng trong HTML được tạo bằng thẻ `table`. Mỗi hàng của bảng là thẻ `tr`, ô tiêu đề là thẻ `th` và ô dữ liệu là thẻ `td`. Thuộc tính `colspan` dùng để gộp nhiều cột, thuộc tính `rowspan` dùng để gộp nhiều hàng.

```
<table>
  <tr><th>Tên</th><th>Điểm</th></tr>
  <tr><td>An</td><td>8.5</td></tr>
  <tr><td>Bình</td><td>7.0</td></tr>
</table>
```

2. Form nhập liệu

Form là thành phần dùng để thu thập dữ liệu người dùng, được tạo bằng thẻ `form`. Thuộc tính `action` chỉ địa chỉ nhận dữ liệu, thuộc tính `method` chọn phương thức GET hoặc POST. Thẻ `input` có nhiều kiểu như `text`, `email`, `password`, `checkbox`, `radio` và `submit`.

```
<form action="/dang-ky" method="post">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>
  <label for="pw">Mật khẩu</label>
  <input type="password" id="pw" name="password">
  <button type="submit">Đăng ký</button>
</form>
```

3. Thẻ label và các thành phần khác

Thẻ `label` dùng để gắn nhãn cho ô nhập liệu: thuộc tính `for` của `label` trùng với `id` của `input`, bấm vào nhãn sẽ đưa con trỏ vào ô nhập. Ngoài ra còn có `textarea` (nhập nhiều dòng), `select` kèm `option` (danh sách chọn). Thuộc tính `required` bắt buộc nhập, `placeholder` hiện gợi ý.

4. GET và POST

GET đưa dữ liệu lên địa chỉ URL, phù hợp để tìm kiếm; POST gửi dữ liệu trong phần thân của yêu cầu, phù hợp với thông tin nhạy cảm như mật khẩu hoặc tạo dữ liệu mới.

Tóm tắt

- `table`, `tr`, `th`, `td` tạo bảng; `colspan`/`rowspan` gộp ô.
- `form` có `action` và `method`; `input` có nhiều kiểu.
- `label for` gắn với `id` của ô nhập để dễ sử dụng.

Câu hỏi ôn tập và bài tập vận dụng

1. Tạo bảng thời khoá biểu có gộp ô bằng `colspan`.
2. Tạo form đăng ký gồm họ tên, email, mật khẩu và nút gửi.
3. Giải thích khi nào nên dùng GET, khi nào dùng POST.

Form và bảng biểu — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: input không có nhãn hoặc người dùng không biết lỗi ở đâu. Kết nối label với id; type phù hợp hỗ trợ nhập liệu nhưng server vẫn phải kiểm tra dữ liệu.

```
<label for="email">Email</label>
<input id="email" name="email" type="email" required>
<button type="submit">Gửi</button>
```

Bảng dùng cho dữ liệu có hàng/cột, không để bố trí toàn trang. Dùng caption và th cho tiêu đề; nội dung dài có thể cần vùng cuộn ngang có kiểm soát.

Tự luyện: gửi rỗng, nhập sai và dùng Tab/Enter. **Hướng giải:** thông báo chỉ ra trường cần sửa, vẫn giữ dữ liệu người dùng đã nhập; không truyền trạng thái lỗi bằng màu đỏ đơn độc.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development

CHƯƠNG 1 · BÀI 3

1.3. Semantic HTML

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu Semantic HTML và lợi ích của nó
> - Sử dụng các thẻ ngữ nghĩa để dựng bố cục trang
> - Phân biệt thẻ ngữ nghĩa với thẻ div và span

1. Semantic HTML là gì

Semantic HTML là cách dùng thẻ có ý nghĩa (thẻ ngữ nghĩa) để mô tả đúng vai trò của nội dung thay vì dùng `div` cho mọi thứ. Dùng thẻ ngữ nghĩa giúp trang tốt hơn cho công cụ tìm kiếm (SEO), dễ bảo trì và thân thiện với trình đọc màn hình của người khiếm thị.

2. Các thẻ ngữ nghĩa phổ biến

- `header`: phần đầu trang hoặc đầu một khối (logo, tiêu đề).
- `nav`: vùng chứa menu điều hướng.
- `main`: nội dung chính của trang, chỉ có một thẻ `main`.
- `section`: một phần nội dung có chủ đề riêng.
- `article`: nội dung độc lập có thể đứng riêng như bài viết, tin tức.
- `aside`: nội dung phụ như thanh bên, quảng cáo.
- `footer`: phần chân trang hoặc chân khối.

```
<body>
  <header><h1>StudyOnline</h1></header>
  <nav><a href="/">Trang chủ</a> <a href="/khoa-hoc">Khoá học</a></nav>
  <main>
    <article>
      <h2>Bài viết đầu tiên</h2>
      <p>Nội dung...</p>
    </article>
    <aside>Khoá học nổi bật</aside>
  </main>
```

```
  <footer>© 2026 StudyOnline</footer>
</body>
```

3. Thẻ div và span

Thẻ `div` là thẻ khối không mang ý nghĩa, chỉ dùng để nhóm nội dung. Thẻ `span` là thẻ nội dòng không mang ý nghĩa, dùng để định dạng một phần văn bản. Chỉ dùng hai thẻ này khi không có thẻ ngữ nghĩa phù hợp.

4. Một số thẻ ngữ nghĩa khác

`figure` và `figcaption` cho hình minh hoạ kèm chú thích, `time` cho ngày giờ, `mark` để làm nổi bật văn bản, `strong` và `em` để nhấn mạnh.

Tóm tắt

- Thẻ ngữ nghĩa mô tả đúng vai trò nội dung, tốt cho SEO và khả năng tiếp cận.
- Bố cục thường gồm `header`, `nav`, `main`, `section`/`article`, `aside`, `footer`.
- `div` và `span` không mang ý nghĩa, chỉ dùng khi không có thẻ phù hợp.

Câu hỏi ôn tập và bài tập vận dụng

1. Dựng bố cục một trang blog bằng các thẻ ngữ nghĩa thay cho div.
2. Liệt kê ba lợi ích của Semantic HTML.
3. Nêu sự khác nhau giữa `section` và `article`.

Semantic HTML — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: div có onclick nhưng không dùng được bằng bàn phím. Dùng button cho hành động và a có href cho điều hướng; không thay tất cả bằng div vì thuận CSS.

Tự luyện: trang gồm header, nav, main, section và footer; tạo liên kết bỏ qua điều hướng để tới main. **Hướng giải:** heading phản ánh cấu trúc; mỗi control có tên dễ hiểu. Ảnh mang thông tin cần alt mô tả; ảnh thuần trang trí dùng alt rỗng. Không thêm ARIA thừa để thay thế semantic sẵn có.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development

CHƯƠNG 2 · BÀI 1

2.1. Selector & thuộc tính CSS

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu CSS là gì và ba cách nhúng CSS vào HTML
> - Sử dụng selector theo thẻ, class, id và pseudo-class
> - Hiểu độ ưu tiên (specificity) và một số thuộc tính phổ biến

1. CSS là gì

CSS (Cascading Style Sheets) là ngôn ngữ dùng để định dạng giao diện cho trang HTML. HTML mô tả cấu trúc nội dung, còn CSS quy định màu sắc, phông chữ và bố cục. Có ba cách nhúng CSS: inline trong thuộc tính `style`, internal trong thẻ `style` và external qua file `.css`. Cách được khuyến khích là external CSS, liên kết bằng thẻ `link` đặt trong phần `head`.

```
<link rel="stylesheet" href="style.css">
```

2. Selector

Selector là mẫu dùng để chọn phần tử HTML cần áp dụng kiểu. Selector theo tên thẻ viết trực tiếp tên thẻ, ví dụ `p` hoặc `h1`. Selector class bắt đầu bằng dấu chấm, ví dụ `.card`; selector id bắt đầu bằng dấu thăng, ví dụ `#header`. Pseudo-class chọn phần tử theo trạng thái, viết sau dấu hai chấm, ví dụ `:hover` áp dụng khi rê chuột lên.

```
h1 { color: #1d4ed8; }
.card { padding: 16px; border: 1px solid #ddd; }
#header { background: #0f172a; }
a:hover { text-decoration: underline; }
```

3. Độ ưu tiên

Độ ưu tiên (specificity) quyết định quy tắc nào được áp dụng khi nhiều quy tắc cùng chọn một phần tử. Thứ tự ưu tiên tăng dần là selector thẻ, selector class, selector id; style inline mạnh hơn cả. Khi hai quy tắc có cùng độ ưu tiên, quy tắc viết sau sẽ được áp dụng.

4. Thuộc tính thường dùng

Thuộc tính `color` đổi màu chữ, `background-color` đổi màu nền. `font-size` đặt cỡ chữ, `font-family` chọn phông chữ. Đơn vị `px` là đơn vị tuyệt đối; `rem` là đơn vị tương đối theo cỡ chữ của phần tử `html`. Thuộc tính `display` điều khiển cách phần tử hiển thị: phần tử block như `div` và `p` luôn bắt đầu trên dòng mới, phần tử inline như `span` và `a` nằm trên cùng dòng, `display: none` ẩn hoàn toàn phần tử.

Tóm tắt

- CSS định dạng giao diện; nên dùng file CSS ngoài.
- Selector: thẻ, `.class`, `#id`, `:hover`; id mạnh hơn class, class mạnh hơn thẻ.
- `display: none` ẩn phần tử; đơn vị `rem` tương đối theo cỡ chữ gốc.

Câu hỏi ôn tập và bài tập vận dụng

1. Tạo file style.css định dạng tiêu đề, đoạn văn và liên kết cho trang của bạn.
2. Tạo class `.nut` làm nút bấm có đổi màu khi rê chuột.
3. Giải thích điều gì xảy ra khi một phần tử có cả style theo class và theo id.

Selector & thuộc tính CSS — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: CSS "không ăn" vì selector không khớp hoặc bị quy tắc khác ghi đè. Kiểm tra Styles/Computed, nguồn stylesheet tải được và tính kế thừa trước khi thêm !important.

```
.course-card { color: #172033; padding: 1rem; }
.course-card:focus-within { outline: 2px solid #2563eb; }
button:disabled { cursor: not-allowed; opacity: .65; }
```

Tự luyện: tạo hai card, đổi một class và giải thích quy tắc nào áp dụng. **Hướng giải:** phân biệt class, id và pseudo-class; khai báo biến CSS cho màu dùng chung. Trạng thái focus không được xóa mà không có cách hiển thị thay thế.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development

CHƯƠNG 2 · BÀI 2

2.2. Box Model & Flexbox

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu Box Model và các thuộc tính margin, padding, border
> - Sử dụng box-sizing: border-box
> - Dựng bố cục một chiều bằng Flexbox

1. Box Model

Box Model là mô hình coi mỗi phần tử HTML như một hộp chữ nhật. Hộp gồm bốn lớp từ trong ra ngoài: content, padding, border và margin. Padding là khoảng đệm bên trong viền, margin là khoảng cách bên ngoài viền.

```
.the {
  width: 300px;
  padding: 16px;
  border: 2px solid #94a3b8;
  margin: 24px auto;
}
```

2. Thuộc tính box-sizing

Mặc định `width` chỉ tính phần content, nên thêm padding và border sẽ làm hộp to hơn. Khi đặt `box-sizing: border-box`, chiều rộng đã bao gồm cả padding và border, giúp tính kích thước dễ hơn. Nhiều dự án đặt `* { box-sizing: border-box; }` ngay từ đầu.

3. Flexbox

Flexbox là mô hình bố cục một chiều, dùng để sắp xếp các phần tử theo hàng hoặc theo cột. Để bật Flexbox, đặt `display: flex` cho phần tử cha (flex container). Flexbox thường dùng để làm thanh menu, căn giữa phần tử hoặc chia đều khoảng cách.

```
.menu {
  display: flex;
  justify-content: space-between;
```

```
  align-items: center;
  gap: 16px;
}
```

4. Các thuộc tính của Flexbox

Thuộc tính `justify-content` căn các phần tử theo trục chính, `align-items` căn theo trục phụ. Thuộc tính `flex-direction` chọn hướng trục chính là `row` (hàng) hoặc `column` (cột). Thuộc tính `flex-wrap: wrap` cho phép các phần tử xuống dòng khi không đủ chỗ. Ở phần tử con, `flex: 1` cho phép phần tử giãn ra chiếm phần không gian còn trống.

Tóm tắt

- Box Model: content, padding, border, margin.
- `box-sizing: border-box` tính cả padding và border vào chiều rộng.
- Flexbox bố cục một chiều với `justify-content`, `align-items`, `flex-direction`, `flex-wrap`.

Câu hỏi ôn tập và bài tập vận dụng

1. Tạo thanh menu ngang bằng Flexbox, các mục cách đều nhau.
2. Căn giữa một hộp theo cả chiều ngang và chiều dọc bằng Flexbox.
3. Giải thích khác nhau giữa margin và padding.

Box Model & Flexbox — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: phần tử width 100% vẫn tràn vì thêm padding và border. Dùng box-sizing phù hợp và kiểm tra kích thước thực.

```
*, *::before, *::after { box-sizing: border-box; }
.actions { display: flex; flex-wrap: wrap; gap: .75rem; }
.actions > * { min-width: 0; }
```

Flexbox phù hợp sắp xếp theo một trục. Với chữ dài, xét min-width:0 và overflow-wrap thay vì giấu mọi phần tràn.

Tự luyện: đặt tiêu đề rất dài và tăng font. **Hướng giải:** nội dung vẫn đọc được, các nút xuống dòng được, không mất phần cuối do height cố định.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development

CHƯƠNG 2 · BÀI 3

2.3. CSS Grid

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu CSS Grid và khi nào dùng Grid
> - Khai báo lưới bằng grid-template-columns
> - Phân biệt Grid và Flexbox

1. CSS Grid là gì

CSS Grid là mô hình bố cục hai chiều, chia vùng hiển thị thành hàng và cột. Để bật Grid, đặt `display: grid` cho phần tử cha. Grid phù hợp để dựng bố cục tổng thể của trang hoặc lưới các thẻ nội dung.

2. Khai báo cột và hàng

Thuộc tính `grid-template-columns` khai báo các cột, ví dụ `repeat(3, 1fr)` tạo ba cột bằng nhau. Đơn vị `fr` là phần của không gian còn trống trong lưới; `gap` đặt khoảng cách giữa các ô.

```
.luoi {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
```

3. Đặt phần tử vào vùng

Một phần tử có thể chiếm nhiều ô bằng `grid-column: span 2` hoặc được đặt theo đường lưới `grid-column: 1 / 3`. Với bố cục trang, dùng `grid-template-areas` để đặt tên các vùng (`header`, `menu`, `noidung`) rất trực quan.

```
.trang {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-areas:
    "header header"
```

```
    "menu   noidung";
}
.header { grid-area: header; }
```

4. Phân biệt Grid và Flexbox

Flexbox là bố cục một chiều, chỉ sắp xếp theo một hàng hoặc một cột tại một thời điểm. Grid là bố cục hai chiều, điều khiển đồng thời cả hàng và cột. Thường dùng Flexbox cho các thành phần nhỏ như menu, dùng Grid cho bố cục tổng thể của trang. Hai mô hình có thể kết hợp với nhau.

Tóm tắt

- Grid là bố cục hai chiều; bật bằng `display: grid`.
- `grid-template-columns` với `repeat()` và đơn vị `fr` tạo lưới linh hoạt.
- Flexbox cho thành phần nhỏ, Grid cho bố cục tổng thể.

Câu hỏi ôn tập và bài tập vận dụng

1. Tạo lưới 3 cột hiển thị 6 thẻ khoá học.
2. Dựng bố cục trang có header, thanh bên và nội dung bằng `grid-template-areas`.
3. Nêu ba khác biệt giữa Grid và Flexbox.

CSS Grid — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: muốn card tự đổi số cột khi thay đổi vùng chứa. Grid tạo bố cục hai chiều; số cột cần dựa vào bề rộng tối thiểu của nội dung.

```
.courses {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 16rem), 1fr));
  gap: 1rem;
}
```

Tự luyện: thử 1, 2, 7 card và một tiêu đề dài. **Hướng giải:** card không bị ép hẹp hơn nội dung; thứ tự DOM vẫn đúng thứ tự đọc. Tránh dùng thuộc tính order chỉ để sửa cấu trúc HTML đang sai ý nghĩa.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development

CHƯƠNG 3 · BÀI 1

3.1. Media Query

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu Responsive Design và thẻ meta viewport
> - Sử dụng Media Query để thay đổi giao diện theo kích thước màn hình
> - Hiểu khái niệm điểm ngắt (breakpoint)

1. Responsive Design là gì

Responsive Design là kỹ thuật thiết kế giúp trang web hiển thị tốt trên mọi kích thước màn hình, từ điện thoại đến máy tính. Để trang hiển thị đúng tỉ lệ trên điện thoại cần khai báo thẻ meta viewport trong phần `head`. Ảnh co giãn theo khung thường dùng `max-width: 100%`.

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

2. Media Query

Media Query là cú pháp CSS dùng để áp dụng kiểu theo điều kiện của thiết bị, bắt đầu bằng `@media`. Ví dụ `@media (max-width: 768px)` chỉ áp dụng kiểu khi màn hình rộng không quá 768px.

```
.luoi { display: grid; grid-template-columns: repeat(3, 1fr); }

@media (max-width: 768px) {
  .luoi { grid-template-columns: 1fr; }
}
```

3. Điểm ngắt

Điểm ngắt (breakpoint) là độ rộng màn hình mà tại đó bố cục thay đổi. Một cách chia phổ biến: dưới 576px là điện thoại, 576 đến 992px là máy tính bảng, trên 992px là máy tính. Nên chọn điểm ngắt dựa trên nơi nội dung bị vỡ chứ không gắn với thiết bị cụ thể.

4. Đơn vị linh hoạt

Dùng `%`, `rem`, `vw`, `fr` thay cho `px` cố định để bố cục co giãn tự nhiên. Kết hợp Flexbox với `flex-wrap` hoặc Grid với `repeat(auto-fit, minmax(240px, 1fr))` để tự xuống dòng mà ít cần media query.

Tóm tắt

- Cần thẻ meta viewport để trang hiển thị đúng trên điện thoại.
- `@media (max-width: …)` áp dụng kiểu theo độ rộng màn hình.
- Chọn điểm ngắt theo nội dung, dùng đơn vị linh hoạt.

Câu hỏi ôn tập và bài tập vận dụng

1. Làm lưới 3 cột chuyển thành 1 cột khi màn hình nhỏ hơn 768px.
2. Làm thanh menu ngang chuyển thành dọc trên điện thoại.
3. Giải thích vai trò của thẻ meta viewport.

Media Query — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: breakpoint được chọn theo tên thiết bị nhưng nội dung vẫn tràn giữa hai kích thước. Chọn điểm đổi bố cục khi nội dung cần, kiểm tra cả kích thước ở giữa.

```
.layout { display: block; }
@media (min-width: 48rem) {
  .layout { display: grid; grid-template-columns: 14rem 1fr; gap: 1rem; }
}
```

Tự luyện: thử 390px, 768px và 1280px, cả phóng to chữ. **Hướng giải:** không chỉ thu nhỏ ảnh desktop. Thử người dùng muốn giảm chuyển động trước khi thêm animation dài; thông tin chính không phụ thuộc hover.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development

CHƯƠNG 3 · BÀI 2

3.2. Mobile First Design

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu nguyên tắc thiết kế Mobile First
> - Viết CSS cho màn hình nhỏ trước rồi mở rộng bằng min-width
> - Ưu tiên nội dung quan trọng trên điện thoại

1. Mobile First là gì

Mobile First là cách viết CSS cho màn hình nhỏ trước, sau đó mở rộng cho màn hình lớn. Theo cách này, media query thường dùng `min-width` thay vì `max-width`. Lý do: phần lớn người dùng truy cập bằng điện thoại, và thiết kế cho màn hình nhỏ buộc ta tập trung vào nội dung thiết yếu.

2. Cách viết

CSS mặc định (không có media query) dành cho điện thoại; các media query `min-width` bổ sung bố cục cho màn hình lớn hơn.

```
/* Mặc định: điện thoại, một cột */
.luoi { display: grid; gap: 12px; }

/* Máy tính bảng trở lên: hai cột */
@media (min-width: 768px) {
  .luoi { grid-template-columns: repeat(2, 1fr); }
}

/* Máy tính: ba cột */
@media (min-width: 1024px) {
  .luoi { grid-template-columns: repeat(3, 1fr); }
}
```

3. Nguyên tắc thiết kế cho điện thoại

- Nội dung quan trọng nhất đặt lên đầu, bỏ bớt phần phụ.
- Nút bấm đủ lớn (khoảng 44px) để chạm bằng ngón tay.

- Cỡ chữ tối thiểu 16px để dễ đọc; giãn dòng hợp lý.
- Hạn chế ảnh nặng; tối ưu ảnh để tải nhanh trên mạng di động.

4. So sánh hai hướng

Desktop First viết cho màn hình lớn rồi dùng `max-width` để thu gọn; thường phải ghi đè nhiều quy tắc. Mobile First bắt đầu từ bố cục đơn giản và chỉ thêm khi cần nên CSS gọn hơn và tải nhanh hơn trên điện thoại.

Tóm tắt

- Mobile First viết CSS cho màn hình nhỏ trước, mở rộng bằng `min-width`.
- Ưu tiên nội dung thiết yếu, nút bấm đủ lớn, chữ dễ đọc.
- CSS thường gọn và tải nhanh hơn so với Desktop First.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết lại một bố cục Desktop First sang Mobile First.
2. Tạo trang có thanh menu gập gọn trên điện thoại và trải ngang trên máy tính.
3. Nêu ba lợi ích của Mobile First.

Mobile First Design — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: màn hình nhỏ có quá nhiều nút và sidebar che nội dung. Thiết kế cấu trúc đơn cột dễ đọc trước, sau đó mở rộng. Menu phải mở/đóng được bằng bàn phím, có trạng thái và trả focus hợp lý khi đóng.

Tự luyện: form đăng ký với nhãn dài, lỗi dài, tên khóa dài và trạng thái đang gửi. **Hướng giải:** input không làm cả trang tràn ngang; bàn phím ảo không che hành động chính; nút có vùng chạm dễ dùng. Không giấu thông tin bắt buộc chỉ để ảnh chụp gọn hơn.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development

CHƯƠNG 3 · BÀI 3

3.3. Dự án: Landing Page Responsive

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

Mục tiêu bài học

- Xây dựng landing page hoàn chỉnh và responsive
- Kết hợp HTML ngữ nghĩa, Flexbox, Grid và media query
- Rèn quy trình làm dự án giao diện từ phác thảo đến hoàn thiện

1. Landing page là gì

Landing Page là trang giới thiệu một sản phẩm hoặc dịch vụ với mục tiêu kêu gọi hành động (đăng ký, mua hàng, tải về). Một landing page thường gồm header, phần giới thiệu (hero), tính năng, đánh giá và footer.

2. Bố cục các phần

- **Header**: logo và menu điều hướng, dùng Flexbox để căn đều.
- **Hero**: tiêu đề lớn, mô tả ngắn và nút kêu gọi hành động (CTA).
- **Tính năng**: lưới 3 thẻ dùng Grid; trên điện thoại còn 1 cột.
- **Đánh giá**: vài trích dẫn của học viên.
- **Footer**: liên kết phụ và thông tin liên hệ.

3. Khung HTML mẫu

```
<header class="dau-trang">
  <a class="logo" href="#">StudyOnline</a>
  <nav><a href="#tinh-nang">Tính năng</a> <a href="#lien-he">Liên hệ</a></nav>
</header>
<main>
  <section class="hero">
    <h1>Học lập trình trực tuyến</h1>
    <p>Khoá học chất lượng, học mọi lúc mọi nơi.</p>
    <a class="cta" href="#dang-ky">Bắt đầu miễn phí</a>
  </section>
  <section id="tinh-nang" class="luoi-tinh-nang">...</section>
```

```
</main>
<footer>© 2026 StudyOnline</footer>
```

4. Quy trình thực hiện

Phác thảo bố cục trên giấy, dựng HTML ngữ nghĩa trước, thêm CSS cho màn hình nhỏ, sau đó mở rộng bằng media query. Kiểm tra trên nhiều kích thước bằng công cụ giả lập thiết bị của trình duyệt (F12).

Tóm tắt

- Landing page gồm header, hero, tính năng, đánh giá, footer với nút CTA nổi bật.
- Flexbox cho menu, Grid cho lưới tính năng, media query cho điện thoại.
- Làm theo thứ tự: phác thảo, HTML ngữ nghĩa, CSS Mobile First, kiểm tra nhiều thiết bị.

Câu hỏi ôn tập và bài tập vận dụng

1. Hoàn thiện landing page giới thiệu một khoá học của bạn.
2. Thêm hiệu ứng rê chuột cho nút CTA và thẻ tính năng.
3. Kiểm tra trang trên ít nhất ba kích thước màn hình và chỉnh lại chỗ bị vỡ bố cục.

CHƯƠNG HỌC THÊM · BÀI TỔNG HỢP

Học thêm.Tổng hợp. Học thêm, bài tập và hướng dẫn giải

Tài liệu học tập HTML & CSS Nhập Môn · Nội dung biên soạn trong dự án StudyOnline.

HTML và CSS — Học thêm về khả năng truy cập và responsive

Tài liệu thực hành nối tiếp HTML, form, semantic, Flexbox, Grid và mobile first. Mục tiêu là một trang dùng được bằng chuột, bàn phím và màn hình nhỏ.

Mục tiêu học thêm

- Chọn phần tử theo ý nghĩa và hành vi mặc định
- Giải thích kích thước theo box model
- Tạo bố cục responsive và kiểm tra tràn nội dung

1. HTML có ngữ nghĩa

Một nút thao tác nên là button, một liên kết điều hướng nên là a có href. Dùng div giả nút sẽ phải tự thêm xử lý bàn phím và các trạng thái mà button có sẵn. Dùng heading theo cấu trúc nội dung, không chọn h1 hay h3 chỉ vì kích thước chữ; kích thước được chỉnh bằng CSS.

```
<main>
  <h1>Đăng ký khóa học</h1>
  <form>
    <label for="email">Email</label>
    <input id="email" name="email" type="email" required>
    <button type="submit">Đăng ký</button>
  </form>
</main>
```

Label liên kết với input qua for/id; placeholder không thay thế label vì có thể biến mất khi nhập. required và type=email hỗ trợ kiểm tra trình duyệt nhưng backend vẫn phải kiểm tra dữ liệu. Ảnh truyền đạt nội dung cần alt phù hợp; ảnh trang trí dùng alt rỗng để không gây nhiễu.

2. Box model và kích thước thực

Với content-box, width chỉ tính nội dung; padding và border cộng thêm bên ngoài. Với border-box, width bao gồm cả nội dung, padding và border.

```
*, *::before, *::after { box-sizing: border-box; }
.card {
  width: 300px;
  padding: 20px;
  border: 2px solid #ddd;
}
```

Nếu dùng content-box, chiều rộng ngoài của thẻ trên là 344px, chưa tính margin. Với border-box là 300px. Margin tạo khoảng cách bên ngoài và không nằm trong width. Khi bố cục tràn, dùng DevTools xem hộp thật thay vì thêm overflow:hidden để che lỗi.

3. Grid, Flexbox và mobile first

Flexbox thường phù hợp sắp xếp một hàng hoặc cột. Grid thuận tiện cho bố cục có hàng và cột. Không có quy tắc buộc luôn dùng một loại; chọn theo cấu trúc cần kiểm soát.

```
.courses {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
}
.course { min-width: 0; overflow-wrap: anywhere; }
img { max-width: 100%; height: auto; }
@media (min-width: 48rem) {
  .courses { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
@media (min-width: 72rem) {
  .courses { grid-template-columns: repeat(3, minmax(0, 1fr)); }
}
```

Mobile first bắt đầu từ bố cục đơn giản cho không gian hẹp rồi mở rộng khi đủ chỗ. Breakpoint nên phản ánh lúc nội dung cần thay bố cục, không phải một danh sách thiết bị cố định. Giữ thứ tự HTML hợp lý; đổi thứ tự bằng CSS có thể khiến thứ tự nhìn thấy khác thứ tự đi bằng phím Tab.

4. Form, focus và nội dung dài

Không xóa outline mà không cung cấp dấu hiệu focus thay thế. Thông báo lỗi cần nói trường nào sai và cách sửa, không chỉ đổi màu đỏ. Trạng thái cũng cần chữ hoặc biểu tượng kèm mô tả để không phụ thuộc duy nhất vào màu.

```
button:focus-visible, a:focus-visible, input:focus-visible {
  outline: 3px solid #1d4ed8;
  outline-offset: 3px;
}
```

Thử trang ở độ rộng 320px, zoom 200%, tiêu đề dài và dữ liệu trống. Bảng dữ liệu lớn có thể cần vùng cuộn ngang riêng thay vì làm toàn trang tràn. Không đặt chiều cao cố định cho khối văn bản nếu nội dung có thể dài hơn.

Câu hỏi ôn tập và bài tập vận dụng

1. Dễ: tính chiều rộng ngoài của width=200px, padding=10px, border=1px với hai chế độ. Đáp án: content-box 222px; border-box 200px, chưa tính margin.
2. Trung bình: làm form đăng ký có label, input email, nút submit và focus nhìn thấy. Tự kiểm tra chỉ dùng Tab và Enter; mọi phần tử thao tác phải tới được.
3. Khó: xây landing page khóa học gồm header, danh sách khóa và form; giữ hoạt động với tiêu đề dài và zoom 200%. Kiểm tra không mất nội dung khi đổi từ ba cột sang một cột.

Tiêu chí hoàn thành

Trang có cấu trúc heading rõ; điều hướng dùng liên kết, thao tác dùng nút; ảnh có alt phù hợp; form có label; bàn phím thấy focus; nội dung không bị cắt trên màn hình hẹp. Lưu ảnh chụp và ghi lại kích thước đã thử để chứng minh responsive.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Accessibility/HTML

https://developer.mozilla.org/en-US/docs/Web/CSS/box-sizing

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design

Dự án: Landing Page Responsive — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Bài thực hành: trang giới thiệu khóa học gồm lợi ích, đề cương, giảng viên mẫu và form liên hệ giả lập. Ghi rõ dữ liệu minh họa, không bịa chứng nhận hay lượt học viên như số liệu thật.

Tự luyện: kiểm tra liên kết, form lỗi, bàn phím, ba chiều rộng và mất ảnh. **Hướng giải:** báo cáo ghi trình duyệt/kích thước đã thử, lỗi còn lại và bằng chứng. Ảnh có width/height hợp lý, không tải ảnh khổng lồ chỉ để thu nhỏ bằng CSS. Một checklist đạt phải có tiêu chí cụ thể, không dùng “đẹp” làm kết quả kiểm thử duy nhất.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Learn_web_development